

Step-by-Step Guide: Creating a Forensic Timeline with Plaso

Plaso (formerly log2timeline) turns scattered forensic artifacts into one chronological, filterable timeline. In the CREST CPIA lab it runs inside the `cpia-forensics` analyst container as two commands:

- `plaso-psteal` - collect + parse artifacts into a single `.plaso` store
- `plaso-psort` - read that store and export a filtered, readable timeline

This guide walks through the full workflow using the real EVTX attack samples in `/artifacts/evtx-attack-samples/`. Every command below was run and verified against this environment (plaso version 20260119), so the syntax is correct for what is actually installed.

1. Enter the analyst container

From the host (Voyager), open a shell in the forensics box:

```
docker exec -it cpia-forensics bash
```

You should now be at a Kali-style root prompt inside the container.

2. Understand where your evidence lives

- `/work` your working directory (host-mounted). Output goes here.
- `/artifacts/evtx-attack-samples/` - the sbousseaden EVTX-ATTACK-SAMPLES repo: 278 `.evtx` files organised into MITRE ATT&CK technique folders: Command and Control, Credential Access, Defense Evasion, Discovery, Execution, Lateral Movement, Persistence, Privilege Escalation, plus a root-level `UACME_59_Sysmon.evtx`.

List what is staged:

```
ls /artifacts/evtx-attack-samples/  
ls "/artifacts/evtx-attack-samples/Credential Access/"
```

Note: technique folder names contain spaces. Always quote them in shell commands.

3. Stage a focused subset of artifacts

Working with all 278 files at once is slow. Stage a small, meaningful subset for a focused investigation. Example - a Mimikatz credential-access scenario:

```
mkdir -p /work/plaso-demo/in  
cp "/artifacts/evtx-attack-samples/Credential Access/CA_DCSync_4662.evtx" /work/plaso-demo/in/  
cp "/artifacts/evtx-attack-samples/Credential Access/CA_Mimikatz_Memssp_Default_Logs_Sysmon_11.evtx" /work/plaso-demo/in/  
cp "/artifacts/evtx-attack-samples/Lateral Movement/LM_4624_mimikatz_sekurlsa_pth_source_machine.evtx" /work/plaso-demo/in/  
ls -l /work/plaso-demo/in/
```

4. Build the timeline store with plaso-psteal

`plaso-psteal` parses the inputs and writes the `.plaso` store. Use `--storage_file` for the store path. NOTE: in this version `plaso-psteal` requires an output file argument as well, so pass `-o l2tcsv -w <csv>` even if you only intend to use the store (`psort` is the proper export tool):

```
cd /work/plaso-demo
plaso-psteal --source /work/plaso-demo/in \
--storage_file /work/plaso-demo/timeline.plaso \
-o l2tcsv -w /work/plaso-demo/psteal-out.csv
```

When it finishes you should see a line like:

```
Processing completed.
Storage file : /work/plaso-demo/timeline.plaso
```

Verify the store is a real SQLite database (not text):

```
file /work/plaso-demo/timeline.plaso
# expect: SQLite 3.x database
```

PITFALL: the log line prints a timestamped storage name like 20260830T185414-in.plaso, but the actual file is the one you named with --storage_file. Use YOUR path, not the printed one.

5. Export a full timeline with plaso-psort

Read the store and write a readable CSV. This version uses `-o l2tcsv` (legacy log2timeline CSV, 17 fixed fields) and `-w` for the output file - `csv` / `--output-file` are NOT valid here:

```
rm -f /work/plaso-demo/timeline.csv
plaso-psort /work/plaso-demo/timeline.plaso \
-o l2tcsv \
-w /work/plaso-demo/timeline.csv
```

PITFALL: psort refuses to overwrite an existing file ("Output file already exists"). Delete the target first (`rm -f`) before re-running.

Preview it:

```
head -n 5 /work/plaso-demo/timeline.csv
```

Each row is one parsed event. The columns include date, time, source, sourcetype, user, host, a short description, and the full message.

6. Verify what the timeline actually contains

Before analysing, confirm the parser ran and see which event IDs were recovered. A quick, reliable way is to dump the store as JSON and tally the event identifiers:

```
cd /work/plaso-demo
plaso-psort timeline.plaso -o json_line -w dump.jsonl

python3 -c "
import json, collections
c = collections.Counter()
for line in open('/work/plaso-demo/dump.jsonl'):
    d = json.loads(line)
    c[d.get('event_identifier', d.get('data_type'))] += 1
for k, v in sorted(c.items(), key=lambda x: str(x[0])):
    print(k, v)
"
```

For the three-file Mimikatz subset this returned:

```
11    2    # file system stat entries (fs:stat)
1102  2    # audit log cleared
4624  2    # account logon
4662  6    # directory service access (DCSync)
4672  2    # special privileges assigned to new logon
4688  6    # new process created
fs:stat 12
```

These map directly to the attack: 4662 = DCSync replication abuse, 4624/4672 = privileged logon, 4688 = process creation, 1102 = log clearing (anti-forensics).

7. Filter by event identifier (recommended)

Filter the store to a single attack-relevant event ID. The filter field is `event_identifier` (an integer):

```
cd /work/plaso-demo
rm -f dcsync.csv
plaso-psort timeline.plaso -o l2tcsv -w dcsync.csv "event_identifier == 4662"
wc -l dcsync.csv
# expect: 7 (header + 6 DCSync events)
```

8. Filter by data type (recommended)

To see only Windows event-log records (and exclude file-system stat noise), filter on `data_type`:

```
cd /work/plaso-demo
rm -f evtx-only.csv
plaso-psort timeline.plaso -o l2tcsv -w evtx-only.csv \
    "data_type == 'windows:evtx:record'"
wc -l evtx-only.csv
# expect: 20 (header + 19 evtx records)
```

Other filterable fields (from the JSON dump): `parser` (e.g. `winevtx`), `source_name`, `computer_name`, `record_number`, `timestamp_desc`, `message_identifier`.

9. Filter by time window (recommended)

Narrow to the attack window. The `date` field takes a quoted datetime:

```
cd /work/plaso-demo
rm -f window.csv
plaso-psort timeline.plaso -o l2tcsv -w window.csv \
    "date > '2019-03-18 11:06:20' AND date < '2019-03-18 11:06:45'"
wc -l window.csv
# expect: 11 (header + 10 events inside the window)
```

Combine a time window with an event filter using `AND`:

```
rm -f combo.csv
plaso-psort timeline.plaso -o l2tcsv -w combo.csv \
    "date > '2019-03-18 11:06:20' AND event_identifier == 4688"
wc -l combo.csv
```

10. Search for indicators of compromise

Grep the exported timeline for suspicious values, such as a username, an account domain, or a specific event text:

```
grep -i "user01" /work/plaso-demo/window.csv
grep -i "DCSync\|directory service" /work/plaso-demo/dcsync.csv
```

11. Get the outputs to the host for the report

/work is host-mounted at /home/billy-forrest/labs/cpia-labs/work/, so anything you save to /work inside the container is already on the host at that path. No `docker cp` is needed. The finished files are available at:

```
ls -la ~/labs/cpia-labs/work/plaso-demo/
```

Quick reference

```
# enter analyst box
docker exec -it cpia-forensics bash

# build the store (must also supply an output file in this version)
plaso-psteal --source /work/plaso-demo/in \
             --storage_file /work/plaso-demo/timeline.plaso \
             -o l2tcsv -w /work/plaso-demo/psteal-out.csv

# full export
plaso-psort /work/plaso-demo/timeline.plaso -o l2tcsv -w /work/plaso-demo/timeline.csv

# filter by event id
plaso-psort timeline.plaso -o l2tcsv -w dcsync.csv "event_identifier == 4662"

# filter by data type
plaso-psort timeline.plaso -o l2tcsv -w evtx.csv "data_type == 'windows:evtx:record'"

# filter by time window + event
plaso-psort timeline.plaso -o l2tcsv -w combo.csv \
             "date > '2019-03-18 11:06:20' AND event_identifier == 4688"

# JSON dump for programmatic analysis
plaso-psort timeline.plaso -o json_line -w dump.jsonl
```

Pitfalls

- **Wrong command names.** Use `plaso-psteal` and `plaso-psort`, NOT `psteal` / `psort` / `log2timeline` (not installed).
- **Wrong output format.** Use `-o l2tcsv` (and `-o json_line` for JSON). `csv` is NOT a valid format name and errors out.
- **Wrong output flag.** Use `-w` (or `--write`). `--output-file` is not recognised in this version.
- **psort will not overwrite.** Delete the target CSV with `rm -f` before re-running, or you get "Output file already exists".
- **Wrong storage path.** After `psteal`, use the path you gave to `--storage_file`, not the timestamped name printed in the log.

- **Wrong filter field.** `source` is not a filterable field for these events. Filter on `data_type`, `event_identifier`, `parser`, or `date`. Quote string values with single quotes inside the double-quoted filter.
- **Never cat a .plaso store.** It is a binary SQLite database. Read it via `plaso-psort` or `file` to confirm its type.
- **Time zones.** Artifact timestamps may be UTC or local. The EVTX samples here are UTC. Keep the time zone consistent across sources or your chronology drifts.
- **Zero events in the store.** Almost always the wrong `--source` path or an unsupported file type. Re-check the path.
- **Folder names with spaces.** Quote technique folder names, e.g. `"/artifacts/evtx-attack-samples/Credential Access/"`.